

Linkage Mechanism Design Domain for DeepXube

Design Document

J.C. Vaught

April 23, 2026

1 Overview

This document specifies a *Linkage Mechanism Design* domain for the DeepXube framework. The puzzle is to transform a planar four-bar linkage so that its coupler curve matches a given target curve. The state encodes the discretized linkage geometry, actions increment or decrement individual link parameters, and the goal is a discretized coupler curve. Because forward kinematics couples all parameters nonlinearly, the problem is non-decomposable and well-suited to learned heuristic search.

2 State Representation

A planar four-bar linkage is defined by four link lengths (a, b, c, d) , one coupler extension length p , and one coupler extension angle β . The ground link d is fixed along the x -axis between the origin $(0, 0)$ and $(d, 0)$. This gives six continuous parameters. Each parameter is uniformly discretized into N_s steps over a physically meaningful range.

The ranges are $a, b, c, d \in [0.5, 5.0]$ and $p \in [0.0, 3.0]$ for lengths, and $\beta \in [0, 2\pi)$ for the coupler angle. With $N_s = 64$ steps per parameter, each parameter maps to an integer in $\{0, 1, \dots, 63\}$ and fits in `int8`. The state array is

```
state = np.array([a_idx, b_idx, c_idx, d_idx, p_idx, beta_idx], dtype=np.int8) (1)
```

with shape $(6,)$. The continuous value for parameter i is recovered as $v_{\min,i} + (\text{idx}_i + 0.5) \cdot \Delta_i$ where $\Delta_i = (v_{\max,i} - v_{\min,i})/N_s$.

The state class stores the `int8` array, with hashing via `.tobytes()` following the established pattern in the framework's `ArmState` and `HanoiState` implementations.

3 Goal Representation

The target coupler curve is encoded as $N_w = 64$ waypoints sampled at uniform crank-angle increments around the full rotation $\theta \in [0, 2\pi)$. Each waypoint's (x, y) coordinates are discretized into spatial bins over the workspace $[-8, 8] \times [-8, 8]$ with $N_b = 64$ bins per axis, yielding `int8` values.

```
goal = np.array([x_bin_0, y_bin_0, x_bin_1, y_bin_1, ...], dtype=np.int8) (2)
```

with shape (128,) (64 waypoints $\times$ 2 coordinates). A special sentinel value -1 marks waypoints where the linkage cannot complete full rotation (non-Grashof configurations), indicating that the coupler point is undefined at that crank angle. This allows the goal to encode both rotatability class and curve shape.

4 Actions

Each action increments or decrements one of the six parameters by one discrete step:

$$\text{actions} = \{(\text{param_idx}, \text{delta}) : \text{param_idx} \in \{0, \dots, 5\}, \text{delta} \in \{-1, +1\}\} \quad (3)$$

This gives $6 \times 2 = 12$ total actions. All actions are state-independent (the same 12 actions are always available), so the domain inherits from `ActsEnumFixed`. Parameter indices are clamped to $[0, N_s - 1]$ rather than wrapping, except for β which wraps modulo N_s . Actions that would leave the parameter unchanged (clamped at a boundary) still cost 1.0 but produce no state change, matching the NPuzzle convention for invalid-but-legal moves.

5 Transition Dynamics

Applying action `(param_idx, delta)` to state s produces state s' identical to s except at position `param_idx`, where the value is clamped or wrapped by `delta`. The transition cost is always 1.0.

The domain is *non-decomposable* because the coupler curve depends on all six parameters simultaneously through the forward kinematics equations. Changing the crank length a alters the range of achievable crank angles, which changes the set of valid waypoints. Changing the coupler length b shifts the entire curve vertically and horizontally. Changing the ground link d rescales the workspace. The coupler extension (p, β) applies a nonlinear transformation that rotates and stretches the curve in a configuration-dependent manner. Consequently, no parameter can be solved independently; improving the match at one waypoint by adjusting one parameter generically worsens the match at other waypoints.

6 Is-Solved Check

The current linkage’s coupler curve is computed via forward kinematics (Section 7) and discretized into the same $N_w \times N_b$ binning as the goal. The state is solved if and only if every non-sentinel goal waypoint’s $(x_{\text{bin}}, y_{\text{bin}})$ matches the corresponding state waypoint exactly:

$$\begin{aligned} \text{is_solved} = \forall i \in \{0, \dots, N_w - 1\} : \\ (\text{goal}[2i] = -1) \vee (\text{curve}[2i:2i+2] = \text{goal}[2i:2i+2]) \end{aligned} \quad (4)$$

The bin width of $16.0/64 = 0.25$ length units provides implicit tolerance. Two curves that agree within ± 0.125 units at every waypoint are considered matching. This is deliberately coarse; finer bins ($N_b = 128$) can increase difficulty.

7 Forward Kinematics

Given continuous parameters (a, b, c, d, p, β) and crank angle θ , the coupler point position is computed via the standard four-bar loop-closure equations. The diagonal f from the crank pin to the ground-link pin satisfies $f^2 = a^2 + d^2 - 2ad \cos \theta$. The angle γ at the diagonal satisfies $\cos \gamma = (b^2 + f^2 - c^2)/(2bf)$. If $|\cos \gamma| > 1$, the linkage cannot assemble at this crank angle and the waypoint is marked sentinel. Otherwise, the coupler point is offset from the floating link by (p, β) in the coupler-link frame.

This computation is $O(N_w)$ trigonometric evaluations per state. With $N_w = 64$, a single forward-kinematics call takes approximately $15 \mu\text{s}$ in vectorized NumPy (batched across waypoints). During A* search expansion with 12 actions, 12 FK calls are needed per node, totaling $\sim 0.2 \text{ ms}$ per expansion. This is comparable to the robotic arm domain’s per-expansion cost and well within the framework’s performance envelope.

8 Visualization

The visualization uses a two-panel layout on a #ECECEC background. The left panel shows the *animated linkage mechanism*: the ground link drawn as a thick bar in Congaree (#1F414D), the crank in Garnet (#73000A), the coupler in Atlantic (#466A9F), and the follower in Horseshoe (#65780B). Pivot joints are rendered as filled circles in 90% Black (#363636). The coupler point traces its path as a thin line in Atlantic with $\alpha = 0.4$ as the crank rotates. The current coupler-point position is marked with a larger diamond in Atlantic.

The right panel shows the *curve comparison*: the target coupler curve as a thick dashed line in Rose (#CC2E40) and the current linkage’s curve as a solid line in Atlantic (#466A9F). Matched waypoints (where bins agree) are marked with small Horseshoe (#65780B) dots. Mismatched waypoints are marked with Rose (#CC2E40) crosses. A status bar at the bottom reports the number of matched waypoints (e.g., “58/64 waypoints matched”) in Horseshoe if solved, Rose otherwise.

For GIF generation, the animation sweeps the crank through $[0, 2\pi)$ over 60 frames, simultaneously drawing the linkage in the left panel and progressively revealing the traced curve in the right panel. Each frame renders in under 50 ms. At the end of the sweep, a 10-frame hold shows the final comparison. This produces a 2–3 second loop at 30 fps that clearly communicates both the mechanism’s motion and the curve-matching objective.

9 Training Considerations

The domain inherits from `GoalStartRevWalkable` and `ActsEnumFixed`. To sample goal-state/goal pairs, a random valid linkage state is generated (six uniform random `int8` values), its coupler curve is computed via FK, and the curve is binned to produce the corresponding goal. Linkages that fail the Grashof condition still produce valid (partial) curves with sentinel waypoints, so no rejection sampling is needed.

The `random_walk_rev` method simply calls `random_walk` since all actions are trivially reversible: incrementing a parameter is reversed by decrementing it, following the same pattern as the Hanoi and NPuzzle domains via the `ActsRev` mixin.

Scramble depths of 10–50 steps are appropriate. At depth 10, the linkage parameters differ from the goal state by at most 10 discrete increments spread across six parameters, producing moderately similar curves. At depth 50, parameters can differ by up to 8 steps each on average, yielding substantially different curves. The state space size is $64^6 \approx 6.87 \times 10^{10}$, comparable to the

3 × 3 Rubik’s Cube (4.33×10^{19} states but only $\sim 10^7$ reachable from a given scramble depth). The branching factor of 12 is modest, making A* feasible with a good heuristic.

The neural network input uses the `HasFlatSGIn` mixin. The state contributes 6 values one-hot encoded to depth 64, and the goal contributes 128 values one-hot encoded to depth 64, for a total flat input dimension of $(6 + 128) \times 64 = 8,576$. A `resnet_fc.2000H_4B` architecture is a reasonable starting point.

10 Challenges and Tradeoffs

Grashof condition violations. Many random linkage configurations are non-Grashof, meaning the crank cannot complete a full rotation. The coupler curve then has undefined segments. The sentinel encoding in the goal handles this gracefully, but it means the space of valid full-rotation linkages is a strict subset of the state space. The heuristic must learn that transitioning between Grashof and non-Grashof configurations can require traversing narrow corridors in parameter space.

Circuit and branch defects. A four-bar linkage generically has two assembly modes (“open” and “crossed”), producing two distinct coupler curves for the same parameters. The FK computation must commit to one branch. This document specifies the open-mode branch by choosing the positive root of the assembly equation. If the target curve was generated from the crossed mode, no state can match it under the open-mode FK, making the problem unsolvable. The `sample_goalstate_goal_pairs` method avoids this by generating targets from the same branch used in FK.

Singularities. When the diagonal length f equals $b + c$ or $|b - c|$, the linkage passes through a dead-center configuration where the assembly angle is 0 or π . Near these configurations, small parameter changes cause large jumps in the coupler curve, creating steep gradients in the cost-to-go landscape. The learned heuristic must capture these discontinuities.

Symmetries. Scaling all link lengths by a constant factor produces a geometrically similar coupler curve. Reflecting the linkage about the x -axis produces a mirror curve. These symmetries are not explicitly broken by the discretization, so the heuristic may encounter equivalent states at different locations in parameter space. Data augmentation during training (flipping $\beta \rightarrow 2\pi - \beta$) could improve generalization.

Computational cost of `is_solved`. Every call to `is_solved` requires a full FK computation ($\sim 15 \mu\text{s}$). During A* search, this is called once per expanded node. In the robotic arm domain, collision checking has a similar per-call cost, so this is within acceptable bounds.